Some time later, the marketing department insists on obtaining full demographic data for all your users, including height in centimeters, zip code, and handedness. (Handedness? Sure, let's say your company makes golf clubs.) And the database administrator insists that means that all demographic categories must be required in the database. Now the test looks like this:

```
it "can tell which user is older" do
  eldest_user = User.create!(date_of_birth: '1971-01-22',
    email: "eldest@example.com", password: "password",
    height: 185, zip code: "60642", handedness: "left")
  youngest_user = User.create!(date_of_birth: '1973-08-31'
    email: "youngest@example.com", password: "password",
    height: 178, zip code: "60642", handedness: "ambidextrous")
  expect(User.eldest).to eq(eldest_user)
  expect(User.youngest).to eq(youngest_user)
end
```

This is starting to get out of control. Now you need to type three lines of text in a test just to create a single user; additionally, it's nearly impossible to pick out of this data the one attribute—date of birth—that is relevant for the test.

Not only that, but this problem happens every time a user is created in a test and every time a new validation is added to users. In other words, all the time.

I'd like a way to specify a known valid object so that there is at most one place to update when new validations get created. Fixtures and factories are two different mechanisms for solving this problem.

Fixtures

Rails has always made it easy to manage a database just for test data, which is automatically cleared between tests. (While there's no denying this is tremendously useful, it has also lulled all of us into feeling that a test that touches the database—a huge third-party dependency—is somehow a *unit* test.) One of the most valuable ways in which Ruby on Rails has supported automated testing is through the use of easily created data that is accessible to all the tests in your system, no matter when or where you write those tests, using *fixtures* specified in a YAML file. (YAML is a file format. The acronym stands for *YAML Ain't Markup Language*.[1]) It's sometimes hard for an experienced Rails programmer to remember just how exciting the YAML fixtures used to seem. You can just set up data once? In an easy format? And it's always there? Amazing.

1. http://www.yaml.org